

SQL 注入漏洞报告（Lab2：登录绕过）

1. 漏洞名称

SQL Injection（SQL 注入）— 登录认证绕过

2. 漏洞等级

高危（High）

3. 漏洞描述

该网站登录功能存在 SQL 注入漏洞。攻击者可以在用户名输入框中插入恶意 SQL 语句，改变后端身份验证查询逻辑，从而绕过密码验证，直接以管理员身份登录系统。

4. 漏洞位置

请求方式：POST /login

漏洞参数：username（用户名输入框）

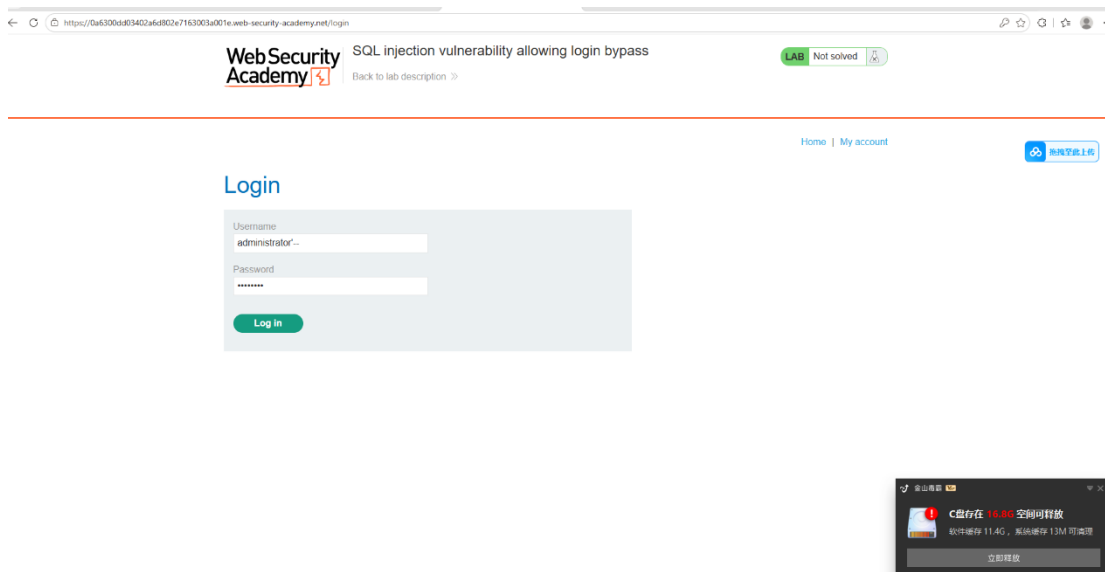
5. 漏洞复现过程

第一步：正常请求

访问 /login 页面，尝试输入普通账号密码登录，系统正常验证，未授权无法进入后台。

第二步：构造 Payload 并注入（截图证据）

在用户名输入框输入注入语句，密码可以随意输入（不校验）。



截图 1：浏览器输入 Payload 状态

说明：-- 是 SQL 注释符，用于注释掉后续的密码校验逻辑。

第三步：漏洞验证与结果（截图证据）

点击登录，系统成功绕过验证，直接跳转至管理员后台。



截图 2：成功登录管理员后台

说明：未输入正确密码，仅凭借 SQL 注入成功登录系统，获取管理员最高权限。



截图 3：实验室完成确认

说明：靶场服务器确认漏洞攻击验证成功。

第四步：漏洞原理

后端原本的 SQL 查询语句逻辑如下：

sql

```
SELECT * FROM users WHERE username='[输入的用户名]' AND password='[输入的密码]'
```

当我们输入 Payload 后，拼接的 SQL 变成了：

sql

```
SELECT * FROM users WHERE username='administrator' -- ' AND password='随便输'
```

- 原 password 的校验逻辑被 -- 成功注释掉，导致 SQL 只校验了用户名 administrator，而不再校验密码。
- 数据库返回管理员用户信息，程序直接通过认证。

6. 漏洞影响

攻击者可利用此漏洞：

- 无需密码登录任意已知用户名的账号。
- 直接获取 **管理员（Administrator）** 权限，接管整个网站后台。
- 进行数据窃取、恶意篡改等操作。

7. 修复建议

① 使用参数化查询 / 预编译语句（最关键）

绝对不要拼接 SQL 字符串：

✗ 错误写法：

sql

```
SELECT * FROM users WHERE username='[输入]' AND password='[输入]'
```

✓ 正确做法（使用数据库驱动提供的参数绑定接口）。

② 密码采用 Hash 加密存储，并在程序逻辑层面进行比对。

③ 限制登录失败次数，防止自动化枚举。

8. 工具

- Firefox / Chrome 浏览器（发送请求）
- PortSwigger Web Security Academy（靶场平台）